

Introduction to Neural Networks

Dr. Osita Onyiah

Logistic Regression : Classification algorithm :

Quick Take on deep learning : Subject of Machine learning

Uses :

- Computer Vision
- Natural Language Processing
- Speech Recognition

Deep learning is extremely computationally expensive.

- People have had to find techniques to parallelize the code.
- Data available has been growing exponentially since the internet bubble.
- This type of algorithm is well suited when there is a plethora of data.
 - models are very flexible.
 - The more data available the better the algorithm can learn salient features within the data.
- Algorithms:

① Logistic Regression

Goal : Find cats in images (binary classification)

If there is a cat in the image, we want to output a number close to 1
If there is no cat in the image, we want to output 0.

For now let us just assume that we are dealing with just one cat in the image and no more.



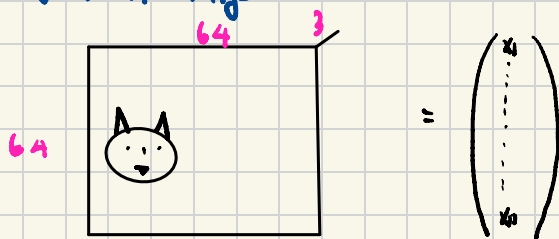
Note that in computer science, images can be described/represented as 2-D matrices.

∴ Assuming the image on the left is a cat image of size 64×64 , we have $64 \times 64 \times 3$ numbers to represent these pixels.

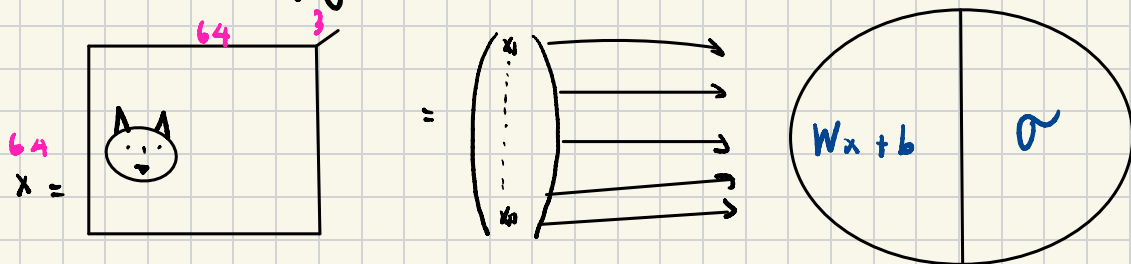
RGB Channel

• Every pixel in an image can be represented by 3 numbers.

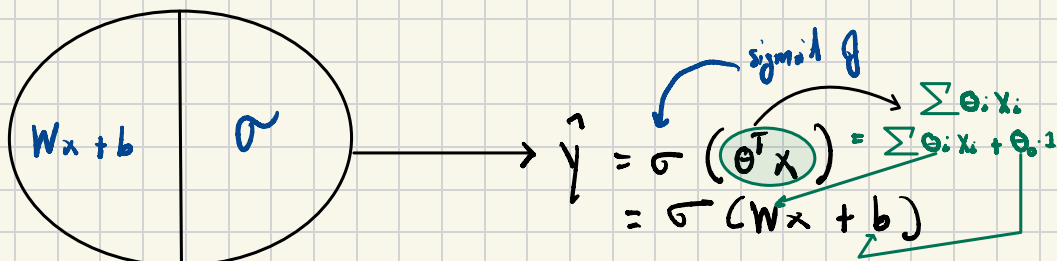
- One representing the red filter, one representing the blue filter and another representing the green filter.
- We will now flatten this image into a vector before using logistic regression to see if there is a cat in the image.



- taking all the numbers in this matrix and flattening it into a vector.
- We can now use our logistic regression because we have a vector input.
- We will now take the vectors and push them into an operation known as the logistic operation, which has one part which is $Wx + b$, where x is the image, and the second part is going to be the sigmoid.



- The sigmoid function is a function that takes a number between $-\infty$ and ∞ , and maps it between 0 and 1. Very common for a classification problem.



Question : What is the shape of the matrix W ?

Answer : $\hat{y} = \sigma(Wx + b)$

Diagram illustrating the dimensions of the matrices and vectors in the equation $\hat{y} = \sigma(Wx + b)$:

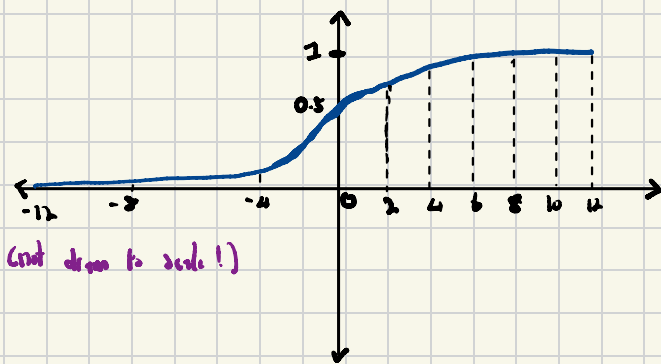
- $\hat{y}$ is a (1×1) scalar.
- σ is a function.
- W is a $(1, 12288)$ matrix.
- x is a $(12288, 1)$ vector.
- b is a $(1, 1)$ scalar.

$\therefore$ We have a row vector as a parameter.

Once we have the model we need to train it. The process of training is that

- initialize parameters W, b (find the right parameters)
- Then optimize them. Find the optimal W, b .
- After finding the optimal $\{W, b\}$, we then to predict.
 $\Rightarrow \hat{y} = \sigma(Wx + b)$ to predict.

Recall that this is a sigmoid function :



$$\begin{aligned}\sigma(x) &= \frac{1}{1 + e^{-x}} \\ &= \frac{e^x}{1 + e^x} \\ &= 1 - \sigma(-x)\end{aligned}$$

- Note that finding the optimal W means defining the loss function, which is the objective.
- In machine learning, it is often the case such that whereby you have a function, that you know you want to find (extreme points) but you do not know the value of its parameters. In order to find them

you are going to use a proxy, that is going to be your **loss function**. If you manage to minimize the loss function you will find the right parameters.

∴ We define a **loss function** that is the **logistic loss** as

$$\mathcal{L} = -[y \log \hat{y} + (1-y) \log(1-\hat{y})]$$

originating from Maximum Likelihood Estimation, starting from a probabilistic model.

∴ The idea is now how do we minimize this function, using a **gradient descent algorithm**. $\Rightarrow$ iteratively compute the derivative of the **loss function** with respect to our parameters.

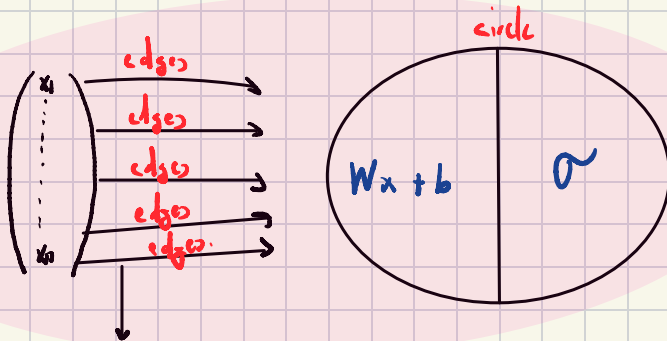
$$\begin{cases} W = W - \alpha \frac{\partial \mathcal{L}}{\partial W} \\ b = b - \alpha \frac{\partial \mathcal{L}}{\partial b} \end{cases}$$

- At every step we will update them to make this loss function go a little smaller.
- At every point you compute the derivative of the loss with respect to your parameters.
- Note that : $\sigma' = \sigma(1-\sigma)$

Q: How many parameters does this Logistic Regression have?

A: 12,288 weights + 1 bias = 12,289 parameters.

Note that every circle has a bias & every edge has a weight.

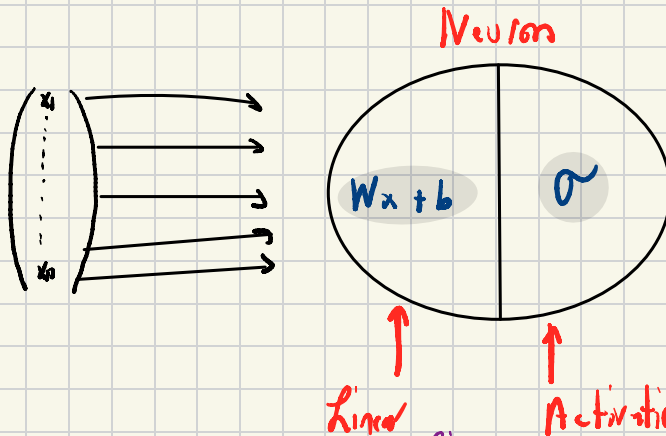


$\sum W_i x_i \Rightarrow$ Every weight corresponds to an edge.

2 Important Equations to know :

① Neuron = Linear + Activation

(A neuron is an operation that has 2 parts, 1) Linear 2) Activation)



Take the output of the linear part & put it into the activation (sigmoid) function in this way however it can be other function.

② Model = architecture + parameters.

one neuron neural network

w, b

- Here we are trying to train a logistic regression in order to be able to be logistic

When people say "we have shipped a model" in industry, what they are saying is that we have found the right parameters with the right architecture.

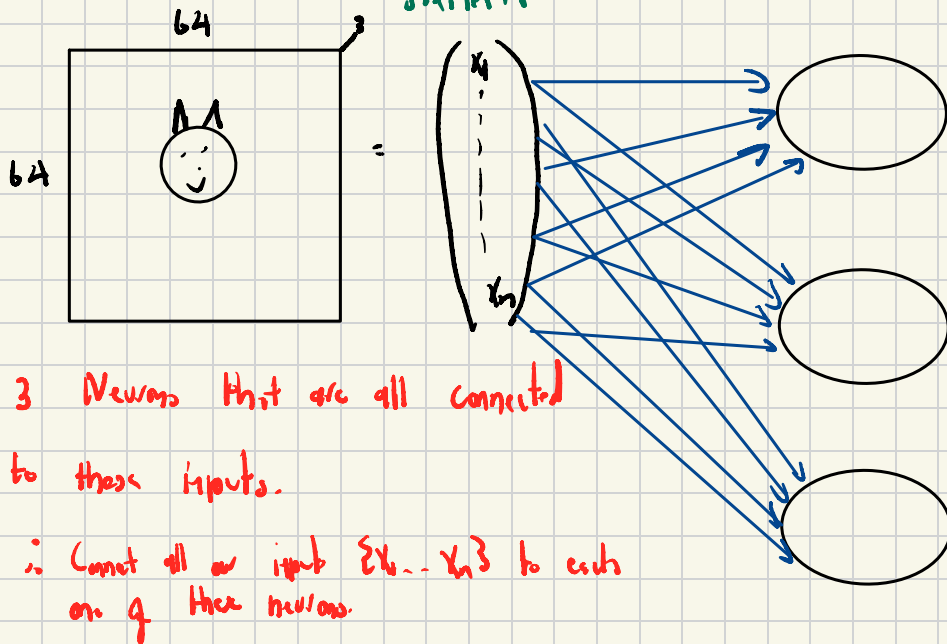
For this series, we will modify the architecture extensively (adding neurons etc) and the parameters will always be called w & b . They will become bigger and bigger because we have more data. We will need more parameters to understand what a cat is.

Goal 2: Find Cat/Kion/Iguana in images

3- types of animals.

⇒ Put 2 more circles/neurons & do the same thing.

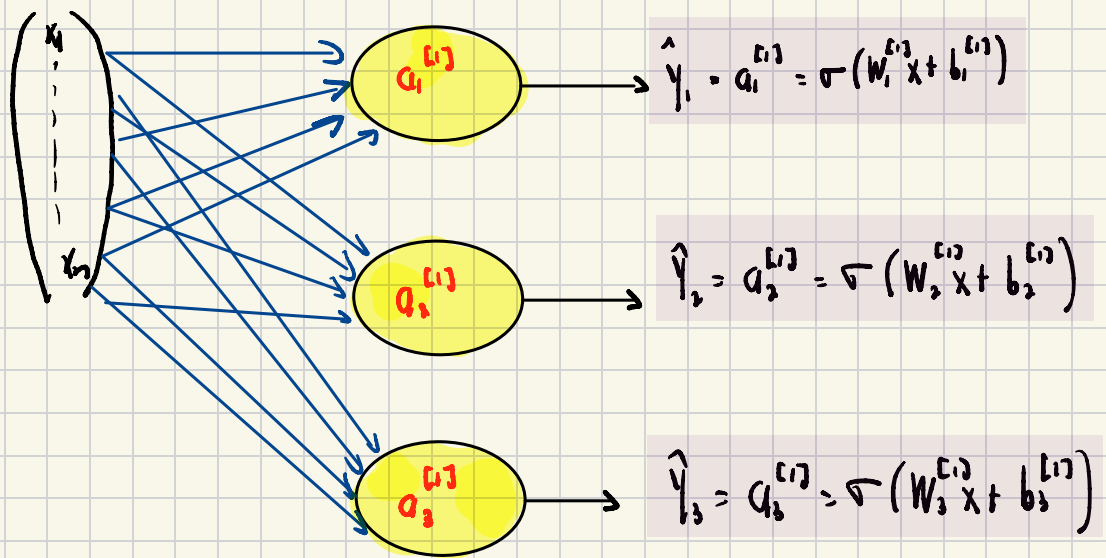
Split it



3 Neurons that are all connected to these inputs.

∴ Connect all our input $\{x_1, \dots, x_n\}$ to each one of these neurons.

We will use a specific set of notation here:



The square brackets represent what we will call the **layer**. If you look at this network, it looks like there is just one layer currently present. In this one layer, the neurons do not communicate with each other.

- The index that is the subscript to the "a" is the number identifying the neuron inside the layer.
- Thus we have $\hat{y}$, and instead of being a single number as it was before, is now a vector of size 3.

Question: How many parameters does this network have?

Answer: 3x what we had before because we added 2 more Neurons.

Note that we are dealing with Binary Classification with images and labels.

- A **label** for an image, which has a cat would probably be a vector with a one and two zeros.
- One should represent the network as follows, with the following label.

Presence $\leftarrow \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \begin{matrix} p(\text{cat}) \\ p(\text{kim}) \\ p(\text{iguma}) \end{matrix} \right\} \text{ Assume we use this scheme to represent the data set.}$

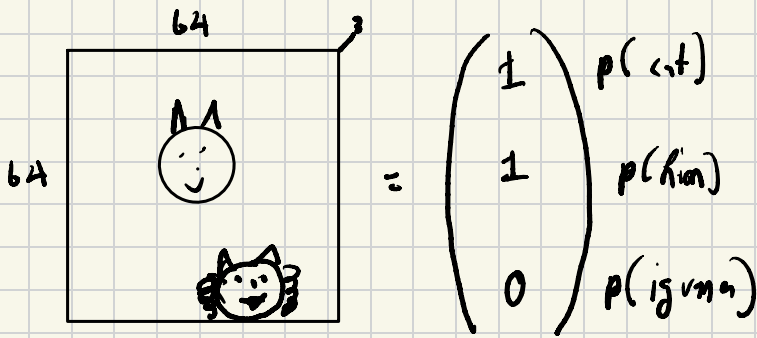
∴ We train this network using the same technique here such that you initialize all of your weights & biases with a starting value, optimize the loss function by using gradient descent, and use $\hat{y}$ to predict.

- The neuron a_1 will be responsible for detecting cats because we decided to label our data set in such a manner. Thus, after training, the neuron a_1 is naturally going to detect cats.
- If we changed the labeling scheme & said the second entry would correspond to the cat, then after training, you will detect 0 that the neuron a_2 is responsible for detecting a cat... etc
- The network will evolve depending on how you label your data set

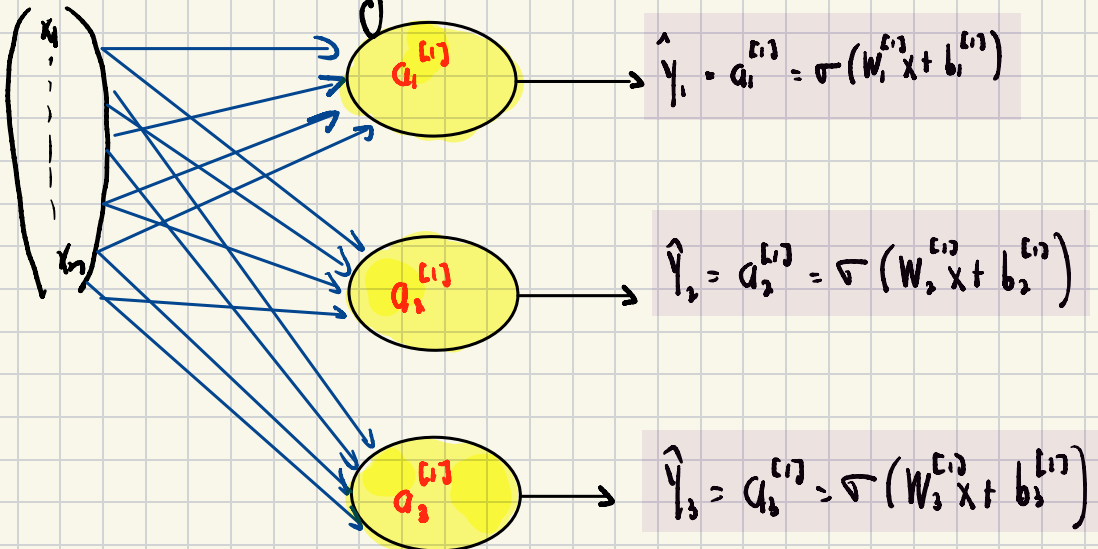
• Do you think that this network could still be robust to different inputs in the same picture?

- This cat now has a friend that is a kim.





- The network only sees $(1, 1, 0)$ & an image. It does not see which "1" corresponds to the cat or the lion. With neural networks you do not need to tell them everything. If you have enough data they will figure them out.
- Even though you will have multiple of those animals either together or alone in a picture, the neuron will know what it's looking for, and will understand that a specific "1" corresponds to a specific animal and yes, it is going to be robust. You just need a lot of data of course to achieve this.
- This system is also going to robust because the 3 neurons are not communicating with each other & thus we can train them



independently from one another. Further, the Sigmoid in y does not depend on the sigmoid in y_2 & does not depend on the sigmoid in y_3 .

$\Rightarrow$ That we can have $\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$ as an output. $\therefore$ You can think

of it as 3 Logistic Regressions. $\therefore$ We will not call this a Neural Network yet.

You can I guess also think of this as a 3-Neuron Network but NOT a Neural Network

Question: Is there a case such that we have a constraint that leads to only one possible outcome? Such that there is no cut and then it is either a cat or a lion.

Answer: Think about health care. There are many models that are made to determine whether or not someone has a skin disease based on cell microscopic images. $\therefore$ You want to classify a specific disease over a huge number of diseases.

$\therefore$ The model we have will still work but it will not be optimal because it's going to take longer to train.

i.e. one disease might be extremely rare and one of the neurons is never going to be trained

i.e. You are walking in a zoo & there is 1 iguana & thousands of lions and cats. The third neuron is never going to be trained

is you would want to start with another model whereby
you put the constraint such that "There is only one
disease that we want to predict."

Then let the model begin with all the Neurons **Training Together**
by **creating** interactions between them.
